

Dynamic Programming (DP)

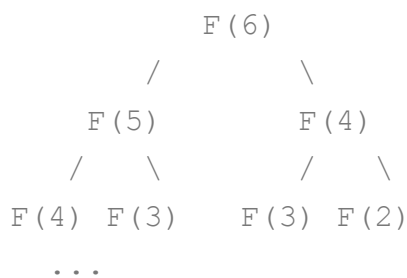
Dynamic Programming (DP) is an algorithmic technique used to solve problems by **breaking them into smaller overlapping subproblems**, solving each subproblem only once, and storing the result for future use.

Instead of solving the same problem repeatedly, **DP remembers the answer**.

Why Dynamic Programming?

Suppose you need to calculate **Fibonacci(6)**.

Without DP:



Notice:

- **F (4)** is calculated multiple times.
- **F (3)** is calculated multiple times.
- Many repeated calculations.

This wastes time.

Dynamic Programming avoids this repetition.

Conditions to Use Dynamic Programming

A problem should have:

1. Optimal Substructure

The optimal solution can be built from optimal solutions of smaller problems.

Example

Shortest path from **A** $\rightarrow$ **D**

can be built using

Shortest path **A** $\rightarrow$ **C** + **C** $\rightarrow$ **D**

2. Overlapping Subproblems

The same subproblem is solved repeatedly.

Example

Fibonacci repeatedly computes

$F(4)$

$F(3)$

$F(2)$

Hence DP stores them.

Two Approaches of DP

There are two main methods.

1. **Memoization (Top-Down)**
 2. **Tabulation (Bottom-Up)**
-

1. Memoization (Top-Down DP)

What is Memoization?

Memoization means

Store already computed answers so they are never calculated again.

The program starts from the main problem and recursively moves downward.

Whenever a value is computed, it is stored in memory.

Next time, it is directly retrieved.

How Memoization Works

Let's calculate

`Fibonacci(6)`

Step 1

Need `F(6)`

`F(6)`

Needs

`F(5)`

`F(4)`

Step 2

Need `F(5)`

Needs

$F(4)$

$F(3)$

Step 3

Need $F(4)$

Needs

$F(3)$

$F(2)$

Continue until

$F(1) = 1$

$F(0) = 0$

Step 4

Store answers

$F(2) = 1$

Dictionary

```
{  
  2: 1  
}
```

Step 5

Compute

$F(3) = 2$

Store

```
{  
  2: 1,  
  3: 2  
}
```

Step 6

Compute

$F(4) = 3$

Store

```
{  
  2: 1,  
  3: 2,  
  4: 3  
}
```

Step 7

Need $F(4)$ again?

Instead of recalculating,

Use

`memo[4]`

Instant answer.

Eventually

$F(6) = 8$

Simple Example (Memoization)

Find the **6th Fibonacci number**.

Python Program

```
memo = {}

def fib(n):
    if n in memo:
        return memo[n]

    if n <= 1:
        return n

    memo[n] = fib(n-1) + fib(n-2)
    return memo[n]

print(fib(6))
print(memo)
```

Output

```
8

{
    2:1,
    3:2,
    4:3,
    5:5,
    6:8
}
```

Time Complexity

Without DP

$O(2^n)$

With Memoization

$O(n)$

Real-Time Example (Memoization)

Online Shopping Discount Calculation

Suppose an e-commerce website calculates the final price after applying many discount rules.

Customer A buys a

Laptop

Final price is calculated.

Stored as

Laptop → ₹52,000

Later,

Customer B also checks the same laptop.

Instead of calculating all discounts again,

the website retrieves

Laptop → ₹52,000

from cache.

This is exactly **Memoization**.

2. Tabulation (Bottom-Up DP)

What is Tabulation?

Tabulation solves the problem **from the smallest subproblems first** and stores every answer in a table (usually an array).

Instead of recursion, it uses iteration.

How Tabulation Works

Again,

Find

`Fibonacci(6)`

Create a table

Index
0
1
2
3
4
5
6

Step 1

Initialize


```
dp[0]=0
dp[1]=1
```

Table

0	1	_	_	_	_	_
---	---	---	---	---	---	---

Step 2

Calculate

```
dp[2]
= dp[1] + dp[0]
= 1 + 0
= 1
```

Table

0	1	1	_	_	_	_
---	---	---	---	---	---	---

Step 3

```
dp[3]
= dp[2] + dp[1]
= 1 + 1
= 2
```

Table

0	1	1	2	_	_	_
---	---	---	---	---	---	---

Step 4

`dp[4]`

`= dp[3] + dp[2]`

`= 2 + 1`

`= 3`

Table

`0 1 1 2 3 _ _`

Step 5

`dp[5]`

`= 3 + 2`

`= 5`

Table

`0 1 1 2 3 5 _`

Step 6

`dp[6]`

`= 5 + 3`

`= 8`

Final Table

Index	0	1	2	3	4	5	6
Value	0	1	1	2	3	5	8

Answer

8

Simple Example (Tabulation)

Python Program

```
def fib(n):  
    dp = [0] * (n + 1)  
  
    if n >= 1:  
        dp[1] = 1  
  
    for i in range(2, n + 1):  
        dp[i] = dp[i-1] + dp[i-2]  
  
    return dp[n]  
  
print(fib(6))
```

Output

8

Time Complexity

Time

$O(n)$

Space

$O(n)$

Real-Time Example (Tabulation)

Climbing Stairs

A person can climb either:

- 1 step
- 2 steps

Find the number of ways to reach the top.

Suppose there are

5 steps

Build a table.

Step	0	1	2	3	4	5
------	---	---	---	---	---	---

Initialize

`Ways[0]=1`

`Ways[1]=1`

Then

`Ways[2]=2`

`Ways[3]=3`

`Ways[4]=5`

Ways[5]=8

Meaning

There are **8 different ways** to climb **5 stairs**.

Python Program

```
def climb_stairs(n):  
    dp = [0] * (n + 1)  
  
    dp[0] = 1  
    dp[1] = 1  
  
    for i in range(2, n + 1):  
        dp[i] = dp[i-1] + dp[i-2]  
  
    return dp[n]  
  
print(climb_stairs(5))
```

Output

8

Memoization vs Tabulation

Feature	Memoization	Tabulation
Approach	Top-Down	Bottom-Up
Uses	Recursion	Iteration (Loops)
Stores Results	Dictionary / Cache	Array / Table
Starts From	Main Problem	Smallest Problem
Stack Usage	Yes	No
Risk of Stack Overflow	Yes (large recursion depth)	No

Feature	Memoization	Tabulation
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(n)$ + Recursion Stack	$O(n)$

Applications of Dynamic Programming

- Fibonacci Number
 - Climbing Stairs
 - Coin Change Problem
 - 0/1 Knapsack Problem
 - Longest Common Subsequence (LCS)
 - Longest Increasing Subsequence (LIS)
 - Matrix Chain Multiplication
 - Edit Distance (Spell Checker)
 - Shortest Path Algorithms (Bellman-Ford, Floyd-Warshall)
 - Resource Allocation and Scheduling
-

Summary

- Dynamic Programming (DP) solves problems by breaking them into **smaller overlapping subproblems** and storing intermediate results.
- Memoization is a **Top-Down** approach that uses recursion and caches computed values to avoid repeated work.
- Tabulation is a **Bottom-Up** approach that starts from the smallest subproblems and builds the solution iteratively using a table.
- Both approaches reduce the time complexity of many problems from **exponential ($O(2^n)$)** to **linear ($O(n)$)**, making them efficient for optimization problems.